Chapter 3 worked solutions

Learning to Align

These written solutions cover Exercises 1, 3, 4, 6, and 7. They follow the arithmetic of a single attention read, explain what changes when the scoring rule or mask changes, and track the dimensions in a batched decoder. The companion notebook covers Exercises 2, 5, and 8–12.

1. Read one row

The decoder is about to predict *coche* in the teaching translation of *the red car*. It has three source annotations, one vector for each source position. In source order these are $\mathbf{h}_1 = (1, 0)$ for *the*, $\mathbf{h}_2 = (0, 1)$ for *red*, and $\mathbf{h}_3 = (1, 1)$ for *car*. A read scales both coordinates of each annotation by its source weight, then adds the three scaled vectors. The first coordinate of the result adds the three first coordinates, and the second does the same for the three second coordinates.

Source position	Annotation	Contribution for $(0.05, 0.10, 0.85)$	Contribution for $(0.05, 0.85, 0.10)$
1, *the*	$(1, 0)$	$0.05(1, 0) = (0.05, 0)$	$0.05(1, 0) = (0.05, 0)$
2, *red*	$(0, 1)$	$0.10(0, 1) = (0, 0.10)$	$0.85(0, 1) = (0, 0.85)$
3, *car*	$(1, 1)$	$0.85(1, 1) = (0.85, 0.85)$	$0.10(1, 1) = (0.10, 0.10)$

For the *coche* row, the two coordinate sums are

$$\mathbf{r}_2 = (0.05 + 0 + 0.85,\ 0 + 0.10 + 0.85) = (0.90,\ 0.95).$$

The *car* annotation contributes to both coordinates. With the second row of weights, which puts the most weight on *red*, the sums become

$$\mathbf{r} = (0.05 + 0 + 0.10,\ 0 + 0.85 + 0.10) = (0.15,\ 0.95).$$

The annotations and their order are fixed. The second row shifts 0.75 of the weight from *car* to *red*, leaving the weight on *the* at 0.05. The first coordinate falls by 0.75 because the *car* annotation has a 1 there and the *red* annotation has a 0. The second coordinate stays at 0.95 because both annotations have a 1 there. More generally, changing the weights can leave a coordinate of the read unchanged when the annotations that exchange weight share that coordinate.

In both cases the three weights sum to one, and each output is a vector with two coordinates. A coordinate can be read as an average of that feature across the three source annotations. The sum across coordinates has a different meaning from the sum across source positions. Here the first context has coordinates that sum to 1.85.

3. Check the normalization axis

In the chapter's invented map, read across a row to find where the decoder looks at one target step. Read down a column to see how much weight a source position receives over the four steps. For example, across the *coche* row the three numbers belong to one prediction. Down the *car* column they come from predictions made at different steps.

Target step	*the*	*red*	*car*	Row sum
el	0.90	0.05	0.05	$0.90 + 0.05 + 0.05 = 1$
coche	0.05	0.10	0.85	$0.05 + 0.10 + 0.85 = 1$
rojo	0.05	0.85	0.10	$0.05 + 0.85 + 0.10 = 1$
EOS	1/3	1/3	1/3	$1/3 + 1/3 + 1/3 = 1$

The column sums use all four rows. Keep the ending entries as exact thirds during addition.

Source position	Column sum
the	$0.90 + 0.05 + 0.05 + 1/3 = 4/3$
red	$0.05 + 0.10 + 0.85 + 1/3 = 4/3$
car	$0.05 + 0.85 + 0.10 + 1/3 = 4/3$

Only the row sums are required to equal one. At each target step, softmax divides each exponential score by the sum of exponential scores over the allowed source positions at that step. Each step therefore has its own denominator. In this map, the equal column totals follow from the chosen numbers. Summing the whole map gives 4 either as four rows of total 1 or as three columns of total 4/3.

Each target step has a fresh total of one to distribute across the source. For instance, two target steps could have rows $(0.60, 0.25, 0.15)$ and $(0.55, 0.35, 0.10)$ over the same source. Both rows sum to one. The first position has the largest weight twice, and the last position contributes to both reads without ever having the largest weight. Row normalization lets each target step distribute its read according to its own scores.

4. Let padding steal a read

The three real source positions have scores $\ln 0.05$, $\ln 0.10$, and $\ln 0.85$. Their exponentials sum to 1. Suppose a rectangular batch stores a fourth annotation $(0, 0)$ with score 0. Softmax uses the fourth score even though the annotation has two zero coordinates. In particular, the exponential of the fourth score is $\exp(0) = 1$, so the denominator becomes $0.05 + 0.10 + 0.85 + 1 = 2$.

Source position	Annotation	Exponential of score	Unmasked weight	Unmasked contribution
the	$(1, 0)$	0.05	0.025	$(0.025, 0)$
red	$(0, 1)$	0.10	0.050	$(0, 0.050)$
car	$(1, 1)$	0.85	0.425	$(0.425, 0.425)$
Padding	$(0, 0)$	1	0.500	$(0, 0)$

The four unmasked weights sum to one, with half of the total assigned to padding. The unmasked read is

$$\mathbf{r}_{unmasked} = (0.025 + 0 + 0.425 + 0,\ 0 + 0.050 + 0.425 + 0) = (0.450,\ 0.475).$$

Now mark the valid positions with the Boolean mask $(True, True, True, False)$. Before softmax, keep each valid score and replace the padding score with $-\infty$. Because $\exp(-\infty) = 0$, the denominator is again $0.05 + 0.10 + 0.85 = 1$. The masked weights are $(0.05, 0.10, 0.85, 0)$, and their contributions give

$$\begin{aligned}\mathbf{r}_{masked} &= (0.05, 0) + (0, 0.10) + (0.85, 0.85) + (0, 0) \\ &= (0.90,\ 0.95).\end{aligned}$$

Zeroing the annotation acts on the weighted sum, which happens after softmax. It makes the direct padding contribution zero. The padding score still changes the denominator and halves each real contribution in this example. The mask acts earlier and removes the padding position from the normalization itself. It is also possible to subtract the largest valid score before exponentiating for numerical stability. The subtraction rescales the numerator and denominator together and preserves the masked distribution.

The row must contain at least one real source position. With an entirely padded source, all allowed exponential mass is zero and the normalization is undefined. The companion implementation raises an error for that case.

6. Remove the nonlinearity

Within one target step t, the decoder supplies the same query $\mathbf{s}_{t-1}$ for every source position. The annotation $\mathbf{h}_j$ changes with the position j. Suppose the scoring rule adds their projections and then immediately multiplies by the score direction $\mathbf{v}_a$. This is the rule obtained by replacing $\tanh$ with the identity function. Matrix multiplication distributes over the addition, so the score becomes

$$\begin{aligned}e_{tj} &= (\mathbf{s}_{t-1}W_s + \mathbf{h}_j W_h + \mathbf{b}_a)\mathbf{v}_a \\ &= \underbrace{(\mathbf{s}_{t-1}W_s + \mathbf{b}_a)\mathbf{v}_a}_{u_t} + \underbrace{(\mathbf{h}_j W_h)\mathbf{v}_a}_{b_j}.\end{aligned}$$

Both u_t and b_j are scalars. The term u_t changes with the query and is shared across the whole score row. The term b_j varies with the annotation. The factor $\exp(u_t)$ appears in both the numerator and denominator of softmax. For a valid position j, it cancels as follows.

$$\alpha_{tj} = \frac{\exp(u_t)\exp(b_j)}{\exp(u_t)\sum_{k\ valid}\exp(b_k)} = \frac{\exp(b_j)}{\sum_{k\ valid}\exp(b_k)} \qquad \text{for valid } j.$$

The source annotations and the validity mask therefore determine the weight vector for this simplified scorer. A different query can move all scores up or down, while their differences and the resulting weights stay the same. The result follows from this particular linear scoring expression with fixed annotations and mask.

To see it with the chapter's identity projections, take $\mathbf{v}_a = (1, -1)^\mathsf{T}$ and zero bias. The annotation terms are $(1, -1, 0)$ in source order. The query $(0.2, -0.1)$ adds 0.3, yielding scores $(1.3, -0.7, 0.3)$. The query $(-0.2, 0.1)$ adds -0.3, yielding $(0.7, -1.3, -0.3)$. Subtracting the shared query term from either score row gives $(1, -1, 0)$. The resulting weight vector is approximately $(0.665241, 0.090031, 0.244728)$ for both queries.

With $\tanh$ restored, the same query is added before the nonlinear function. A shift in a feature near zero can change its $\tanh$ value more than the same shift in a feature whose value is already near saturation. Different annotations can therefore respond differently to the shared query. For the chapter's identity example, the score gap between annotations $(1, 0)$ and $(1, 1)$ becomes $\tanh(s_2 + 1) - \tanh(s_2)$, where s_2 is the second coordinate of the query. The gap is about 0.815966 at $s_2 = -0.1$ and 0.700831 at $s_2 = 0.1$. That change alters the ratio of the two source weights. The nonlinear combination gives the trained scorer a way to change its reads with the query.

7. Follow the shapes

Imagine two color sequences stored in a batch with room for five source positions per example. The decoder produces six prediction steps per example in its rectangular output. Let $B = 2$ be the batch size, $S = 5$ the stored source width, and $T = 6$ the stored target width. The annotation and decoder hidden widths are $H = D = 64$. The internal scoring width is $A = 32$, and the vocabulary size is $V = 9$.

Quantity	Shape	Axes
Original annotations	$(2, 5, 64)$	Batch, source position, annotation feature
Projected annotations	$(2, 5, 32)$	Batch, source position, scorer feature
Scores at one decoder step	$(2, 5)$	Batch, source position
Contexts at one decoder step	$(2, 64)$	Batch, annotation feature
Collected attention weights	$(2, 6, 5)$	Batch, target step, source position
Collected vocabulary logits	$(2, 6, 9)$	Batch, target step, vocabulary entry

The source projection multiplies every annotation by the same 64×32 matrix. It replaces the feature width of 64 with 32 and retains the two axes that tell us which example and source position each vector belongs to. These projected annotations can be stored once for all six decoder steps. The Boolean validity mask has shape $(2, 5)$ and uses the same first two axes as the annotations.

At a single target step the batch of previous hidden states has shape $(2, 64)$. Its projection has shape $(2, 32)$. Insert a length-one source axis to get $(2, 1, 32)$, so each projected query can be added to each of that example's five projected annotations. After $\tanh$, the scorer reduces the 32 features at every source position to one scalar. A projection can produce $(2, 5, 1)$ first. Removing that last length-one axis gives the required $(2, 5)$ score array. Each example has one length-five score row. Masked softmax normalizes over its five source slots, giving a weight array of the same shape with zeros at padding positions.

For the read, those weights scale the five **original** annotations per example. Each context vector has 64 coordinates because the weighted sum removes the source axis and keeps the annotation features. The contexts therefore have shape $(2, 64)$ for the whole batch at one step. The 32 features used to score a source position and the 64 features read from it serve separate roles.

Each decoder step also produces nine logits per example from its updated hidden state. Stacking the six $(2, 5)$ weight arrays and the six $(2, 9)$ logit arrays along the new target-step axis gives $(2, 6, 5)$ and $(2, 6, 9)$. Indexing the weights by batch item and target step selects a distribution over source positions. Indexing the logits by the same pair selects one score per vocabulary token, to be normalized over the vocabulary if probabilities are needed.